Uncertainty-gated selection for block-sparse attention*

A value-of-information view of the SSA-style selector

June 9, 2026

Abstract

Block-sparse attention scales long-context language models by replacing the $O(N^2)$ softmax with a per-query top-k selection over key blocks. This cutoff is myopic: when the k-th and $(k{+}1)$-th blocks are nearly tied in score, the selector commits without spending extra budget, and the dropped block – sometimes the only carrier of an answer hop – is unrecoverable downstream. We propose a value-of-information *router* that measures, for each query, how decisively the top-k cut was made – the score gap between the last kept block and the first rejected one – and, for the queries where that gap is smallest, doubles the kept set. The rule is independent of how block scores are computed and composes with existing scoring backbones. We isolate selector quality from model capability with a *paired recall* metric (the fraction of dense-correct examples the sparse policy also solves) and a diagnostic benchmark, the Pointer-Chase Haystack. On Qwen2.5-14B-Instruct at 32K, $n = 100$, evaluated on top of two scoring backbones – SSA-style mean pooling and Quest's K-min/K-max upper bound – the router lifts RULER NIAH-multikey paired recall from 0.51 $\rightarrow$ 0.63 (mean) and 0.93 $\rightarrow$ 0.98 (Quest, within 2 pp of dense), and LongBench-v2 medium paired recall from 0.40 $\rightarrow$ 0.60 (mean) and 0.60 $\rightarrow$ 0.75 (Quest). A LongBench-v1 negative control localises the regime: the lift activates only when the context-to-budget ratio binds. The fused selection-plus-kernel pipeline runs at 0.76–0.85$\times$ dense wall time at 64K and the crossover widens structurally with context length.

1 Introduction

Long-context language models increasingly use *block-sparse attention* as a drop-in replacement for the $O(N^2)$ softmax. The idea is shared by Quest (Tang et al., 2024), H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), MInference (Jiang et al., 2024), NSA (Yuan et al., 2025), MoBA (Lu et al., 2025), and Subquadratic's SSA (Subquadratic, 2025): a cheap per-query *selector* picks k out of N_B key blocks; exact attention then runs only on the selected set. The selector is the lever – it decides where the model attends. Today it is universally implemented as a *single-shot top-k rule* on a pooled inner product $q \cdot \bar{k}_b$ between the query and a per-block key summary.

This is a myopic decision. When the k-th and $(k{+}1)$-th block are nearly tied, top-k silently breaks the tie and moves on; if the dropped block contained an evidence token, the answer is gone, and no downstream layer can recover it. The failure mode is structural, not noisy: it bites hardest on *multi-hop* and *query-latent* retrieval, where the relevance of a block depends on what was learned earlier in the same forward pass and is not visible to the selector's surface query–key match. The published SSA

*Code, data, and reproduction scripts: https://github.com/ThomasRossi/uncertainty-gated-block-sparse-attention

numbers themselves show this: NIAH multi-key recall degrades 96 → 83 → 68% as the number of keys grows from 2 to 8.

This paper. We treat the top-k cutoff as a *value-of-information* (VoI) decision and add a single layer of policy on top of it. For each Q-tile and head, we compute the normalised cutoff margin

$$\sigma \;=\; \frac{s_{(k-1)} - s_{(k)}}{s_{(0)} - s_{(k)}} \;\in\; [0,1],$$

where $s_{(\cdot)}$ are the sorted block scores. A small σ means the cutoff is high-risk; we then *route* that tile to a 2× expanded `kv_idx` – it gets to attend to more blocks – while confident tiles keep the baseline k_{budget}. The expansion is paid for selectively: we trigger on the bottom q-fraction of tiles per layer, so the average attended set grows by only $1 + q$ blocks per row.

The router is backbone-agnostic. The cutoff margin σ is a function of the sorted block scores; it does not depend on how those scores are computed. Existing selectors differ in their block-scoring backbone – SSA pools keys with a mean ($\bar{k}_b = \frac{1}{B_n}\sum_j k_j$), Quest uses a min/max upper bound ($s = \sum_d \max(q_d k_{b,d}^{\max}, q_d k_{b,d}^{\min})$). The router can sit on top of any of them. This turns the contribution from *a replacement for top-k* into *a generic budget-allocation layer that stacks on whichever scoring backbone is best for the task*. We validate this empirically: **better scoring (Quest) and better budget allocation (router) are orthogonal directions; combining them strictly dominates either alone** on both benchmarks we test.

Contributions.

1. A VoI formulation of the selector cutoff that adds one scalar per tile and one quantile threshold per layer, independent of the block-scoring backbone. No retraining, no extra parameters, no per-row keep tensor.
2. Empirical demonstration that the router composes with at least two distinct scoring backbones (K-mean and Quest's K-min/K-max), and that the combination beats each on its own. Headline at $n = 100$, 32K context: on RULER NIAH multikey, router-on-Quest paired recall **0.98** vs Quest 0.93 vs top-k 0.51 vs dense 1.00 – within 2 pp of dense. On LongBench-v2 medium, router-on-Quest paired **0.75** vs Quest 0.60 vs top-k 0.40 vs dense 1.00.
3. A fused selection-plus-kernel implementation that produces `kv_idx` directly per Q-tile and handles the routed expansion in the same code path, keeping kernel dispatch shape-uniform. All four sparse policies (top-k, router, Quest, router-on-Quest) run on this same kernel; they differ only in the selection step. Wall-time profile crosses dense at 32K and widens at 64K (0.76×–0.85×); the crossover regime is characterised via an Amdahl decomposition of the prefill.
4. A diagnostic benchmark, the *Pointer-Chase Haystack* (PCH), engineered to isolate selector quality from model capability.
5. A negative-control result (LongBench-v1) that pins down *when* the router helps: only when the context-to-budget ratio binds.

2 Background and related work

2.1 Block-sparse attention

A standard decoder layer maps hidden state $h^{(L)} \in \mathbb{R}^{N \times d_{\text{model}}}$ to $h^{(L+1)}$ via

$$h' = h^{(L)} + \text{Attn}\big(\text{LN}(h^{(L)})\big), \qquad h^{(L+1)} = h' + \text{MLP}\big(\text{LN}(h')\big).$$

The attention block, for query position i on head h, computes

$$\mathrm{Attn}(x)_{i,h} \;=\; \sum_{j \le i} \frac{\exp\!\big(q_{i,h} \cdot k_{j,h} / \sqrt{d_{\text{head}}}\big)}{\sum_{j' \le i} \exp\!\big(q_{i,h} \cdot k_{j',h} / \sqrt{d_{\text{head}}}\big)}\; v_{j,h}.$$

Block-sparse attention replaces $\{j \le i\}$ by a small selected subset S_i, chosen per query at inference time on frozen weights.

2.2 The selector landscape

Concrete selectors differ in (a) how they pool keys into a per-block summary and (b) how they score blocks against the query, but **they all reduce, at the per-query selection step, to a top-k rule over a block score**:

- **SSA** (Subquadratic, 2025) (Subquadratic Sparse Attention) – per-Q-tile top-k over mean-pooled keys, $s_b = q \cdot \bar{k}_b$ with $\bar{k}_b = \frac{1}{B_n} \sum_j k_j$. Cheap to compute; mean-pooling blurs single hot keys, which is exactly the multi-key NIAH degradation discussed above.
- **Quest** (Tang et al., 2024) – per-block elementwise $K_b^{\min}, K_b^{\max}$ summaries; score is an upper bound on $\max_{j \in b} q \cdot k_j$, computed coordinate-wise (see Eq. 2). Recovers sharp single-key needles that mean-pooling loses.
- **H2O** (Zhang et al., 2023), **SnapKV** (Li et al., 2024), **MInference** (Jiang et al., 2024) – top-k over learned or attention-history-based block scores.
- **NSA** (Yuan et al., 2025), **MoBA** (Lu et al., 2025) – end-to-end learned gating, but still resolves to a top-k over block scores at the per-query selection step.

What none of these methods do is treat the cutoff as a *decision under uncertainty*: when $s_{(k-1)} \approx s_{(k)}$, the selector commits without spending extra budget. That step is the lever this paper pulls. Because the lever is on the cutoff (not on how s is computed), it composes with any of the scoring backbones above. In the experiments we evaluate it on top of both the SSA-style K-mean backbone and Quest's K-max upper bound.

2.3 Long-context evaluation

The published benchmarks fall into two families. **Synthetic / diagnostic:** RULER (Hsieh et al., 2024) (NIAH, VT), BABILong, MRCR. Designed for controlled stress tests of long-context recall. Their failure modes are interpretable but they are *not* predictive of downstream performance, as HELMET (Yen et al., 2024) has documented. **Real-task:** LongBench (Bai et al., 2023, 2024), HELMET (Yen et al., 2024), NoCha. These cover multi-hop QA, summarisation, code, and so on. LongBench-v2 in particular has a *medium* split with native prompt lengths typically ≥100K words, designed to stress long-context selection. We use **PCH** (diagnostic, ours), **RULER NIAH** (synthetic, standardised), and **LongBench-v1 + v2 medium** (real-task, standardised).

3 Method

We describe the working method end-to-end. Each step is motivated by the previous one and lifts a specific weakness. The full per-equation derivation lives in Appendix A; here we give the path the reader needs to follow to understand the experiments.

3.1 Step 1: block scoring

Keys are grouped into contiguous blocks of $\text{BLOCK}_N = 64$ tokens. The selector at layer L scores each block $b \in \{0, \ldots, N_B - 1\}$ via

$$s^{(L)}[i, h, b] \;=\; \frac{q_i^{(L,h)} \cdot \bar{k}_b^{(L,h)}}{\sqrt{d_{\text{head}}}}, \qquad \bar{k}_b^{(L,h)} \;=\; \frac{1}{\text{BLOCK}_N} \sum_{j \,\in\, \text{block } b} k_j^{(L,h)}. \tag{1}$$

This is the standard pooled-key score used throughout the literature. The information-theoretic content of s is bounded by the rank of the pooled-key matrix; we found this in practice via a spectral diagnostic (effective rank $\approx$ 60–80 at 95% energy in mid-layers), which informs how aggressively the score itself can be compressed but does not change its top-k structure.

Backbone choice. The block-scoring rule above is one of several. Quest (Tang et al., 2024) replaces $\bar{k}_b$ with an elementwise min/max pair $(K_b^{\min}, K_b^{\max})$ and scores

$$s_b^{\text{quest}} \;=\; \sum_{d=1}^{d_{\text{head}}} \max\!\big(q_d \cdot K_{b,d}^{\max},\; q_d \cdot K_{b,d}^{\min}\big), \tag{2}$$

which is an upper bound on $\max_{j \in b} q \cdot k_j$ – it preserves sharp single-key signals that the mean averages away. We treat the choice between Eq. (1) and Eq. (2) as a *backbone hyperparameter*; everything downstream (per-tile selection, σ, the trigger) is identical. The experimental section reports both backbones with and without the router; the headline result is that the gains compose.

3.2 Step 2: per-tile selection

A naive per-row top-k would emit a boolean keep tensor of shape $[B, H, M, N_B]$, which downstream has to be sorted and unioned across rows in a Q-tile – a step whose cost we measured to scale super-linearly in N.

Following the SSA recipe, we group $\text{BLOCK}_M = 64$ consecutive query rows into a *Q-tile* t and make one selection decision per tile, shared by all rows in that tile:

$$\text{tile\_score}[t, h, b] \;=\; \max_{r \,\in\, \text{tile } t} s[r, h, b], \qquad \text{kv\_idx}[t, h] \;=\; \text{top-}k\big(\text{tile\_score}[t, h, \cdot]\big). \tag{3}$$

The sink block 0 and the tile's own block are forced into kv_idx by adding $+\infty$ to their scores *before* the top-k – this avoids a downstream concat-and-dedup. The output is a single integer tensor of shape $[B, H, Q_t, k_{\text{budget}}]$ (where $Q_t = M/\text{BLOCK}_M$), passed straight to the attention kernel.

What this buys. The kernel iterates each tile's kv_idx without any per-row inner mask. No keep tensor is ever materialised. Wall time for selection scales as $O(NN_B/\text{BLOCK}_M)$ (instead of $O(NN_B)$ for per-row).

What this costs. Per-tile selection is coarser than per-row. A needle that one row strongly wants can be outranked by mediocre-everywhere distractors. This shows up as a hit-rate drop on PCH (per-tile is ≈ 0.42 at 64K vs. per-row ≈ 0.97 in earlier configurations). Step 3 spends a controlled amount of budget to claw back the part of this drop attributable to ambiguous cutoffs.

3.3 Step 3: the cutoff is a decision – read its uncertainty

The top-k in Eq. (3) is a decision: keep block $(k-1)$, drop block (k). Its *quality* is naturally measured by how decisive that ranking is. Let $s_{(0)} \geq s_{(1)} \geq \cdots$ be the sorted tile scores for tile t, head h. We define the normalised cutoff margin

$$\sigma[t,h] \;=\; \frac{s_{(k-1)} - s_{(k)}}{s_{(0)} - s_{(k)}} \;\in\; [0,1]. \tag{4}$$

The interpretation is direct:

- $\sigma \to 1$ – the kept set is far above the rejected tail; the cutoff is unambiguous.
- $\sigma \to 0$ – the kept set's last element is tied with the first rejected element; the cutoff is a coin flip.

We compute σ from the same top-$(k{+}1)$ we already need for kv_idx, at no extra asymptotic cost.

Why this is a value-of-information signal. If we were to commit to the top-k, the expected loss from the cutoff decision is bounded by the probability mass the softmax would have placed on the dropped block. When the k-th and $(k{+}1)$-th scores are close, that mass is large; when they are well-separated, it is small. The cutoff margin σ is a cheap, dimensionless proxy for this expected loss. The formal anchor is in Appendix C: under a sub-Gaussian noise model on the block scores, σ is the dimensionless analogue of the asymptotically optimal best-arm-identification exploration index of Garivier and Kaufmann (2016), and the σ-quantile router is $C(\tau)$-times optimal in top-k identification, with $C(\tau) \leq 2$ under Gaussian noise.

3.4 Step 4: aggregate σ across heads

A natural reflex is to gate per-(tile, head) cell. We tested this and it does not work (Appendix B): heads in the same tile are highly correlated, and per-cell gating is dominated by per-head idiosyncratic noise that has no relation to whether the tile is actually ambiguous.

Aggregating across heads at the tile level recovers the signal. We use a uniform mean:

$$\bar{\sigma}[t] \;=\; \frac{1}{H}\sum_{h=1}^{H} \sigma[t,h]. \tag{5}$$

$\bar{\sigma}$ is the tile's overall decision confidence: low when many heads simultaneously face an ambiguous cutoff, high when most heads agree the cutoff is clean.

3.5 Step 5: trigger on the bottom q-fraction

We turn $\bar{\sigma}$ into a binary decision via a per-layer empirical quantile:

$$\text{trigger}[t] \;=\; \mathbb{1}\Big[\bar{\sigma}[t] \;\leq\; \text{quantile}_q\big(\bar{\sigma}[\,\cdot\,]\big)\Big]. \tag{6}$$

q is the only hyperparameter introduced by the router; intuitively it is the per-layer fraction of tiles that are deemed risky enough to spend extra budget on. We use the same q across all layers and find it stable; the working value is $q = 0.40$.

Why a quantile, not an absolute threshold. The absolute scale of σ varies across layers (early layers are more peaked than later layers). A quantile is the simplest layer-conditional normalisation. We did test absolute and per-cell quantile thresholds; they fail (Appendix B).

3.6 Step 6: expand kv_idx uniformly

Triggered tiles get a $\rho\times$ expanded selection (we use $\rho = 2$). To preserve a fixed kernel dispatch shape across triggered and non-triggered tiles, we allocate $[B, H, Q_t, \rho k_{\text{budget}}]$ uniformly and pad non-triggered tiles' extra slots with the kernel's N_B-sentinel:

$$\text{kv\_idx}[t,h] \;=\; \begin{cases} \text{top-}\,\rho k\big(\text{tile\_score}[t,h,\cdot]\big) & \text{if trigger}[t]=1, \\ \text{top-}\,k\big(\text{tile\_score}[t,h,\cdot]\big) \;\|\; [\,N_B,\ldots,N_B\,] & \text{otherwise.} \end{cases} \tag{7}$$

The kernel skips $b = N_B$ on a fast check, so non-triggered tiles cost the same as plain top-k. The end-to-end forward is otherwise unchanged: a standard FlashAttention-style online softmax, iterating each tile's kv_idx instead of all N_B blocks.

Cost accounting. Average kept blocks per row is $k_{\text{budget}}(1 + q(\rho - 1))$. At $q = 0.4, \rho = 2$ this is $1.4\,k_{\text{budget}}$. The wall-time cost is sub-proportional because the extra blocks are concentrated in the most ambiguous tiles, where the kernel's inner loop is anyway not memory-bound. The measured router/topk kernel-time ratio at 32K is ≈ 1.53 and at 64K is ≈ 1.47 (Section 4.6).

4 Experiments

4.1 Setup

Model. Qwen2.5-14B-Instruct, weights frozen (no retraining, no fine-tuning). **Hardware.** A100-80GB via Modal. **Sparse path.** Custom Triton block-sparse attention kernel with fused per-tile selection. **Baselines and policies.**

- *dense* – FlashAttention (Dao et al., 2022) via PyTorch SDPA, the ceiling.
- *top-k* – per-tile top-k on the K-mean backbone (Eq. 1). SSA-style baseline; uses only Step 2.
- *Quest* – per-tile top-k on the Quest K-max upper-bound backbone (Eq. 2). Strong baseline on sharp-needle tasks.
- *router* – Steps 2–6 on top of the K-mean backbone, $q = 0.40, \rho = 2$. Uncertainty-gated expansion only.
- *router-on-Quest* – Steps 2–6 on top of the Quest K-max backbone, same q, ρ. Combines better scoring with budget allocation.

All four sparse policies share the same fused selection-plus-kernel implementation; they differ only in (a) which backbone scores blocks and (b) whether the router expansion is applied. Wall-time and quality numbers are therefore directly comparable across them. **Selector budget.** Fixed at $k_{\text{budget}} = 33$ blocks per tile (NIAH/LB) or 40 (PCH 32K/64K), corresponding to roughly 2K attended tokens per row.

Metrics. We report three quantities.

- **Accuracy** (or task score, e.g. F1): the standard unconditional success rate – fraction of examples the policy answers correctly. This is what the long-context literature usually reports.
- **Paired recall**: *among the examples dense answers correctly*, the fraction the sparse policy also answers correctly. We compute it as $|\{i : \text{dense}_i \text{ correct} \wedge \text{sparse}_i \text{ correct}\}|/|\{i : \text{dense}_i \text{ correct}\}|$. Dense paired recall is 1.0 by construction; a sparse policy with paired recall 1.0 would preserve every dense-correct example. We write n for the total number of examples run and $n_{dc} \leq n$ for the number of those examples dense answers correctly; n_{dc} **is the denominator of paired recall, not** n. The two coincide only when dense achieves 100% accuracy.
- **Hit rate**: fraction of gold-evidence blocks the selector keeps in kv_idx (used on PCH where the gold blocks are known by construction).

Paired recall is not a standard named metric in the long-context literature – Quest, H2O, SnapKV, MInference, NSA, and SSA all report unconditional accuracy. We adopt it because raw accuracy conflates two failure modes: *(i)* the selector dropped a needed block, or *(ii)* the model could not have answered the question even with full attention. Paired conditions on $\{i : \text{dense}_i \text{ correct}\}$, which is the question set where *(ii)* is ruled out by construction; the gap between dense paired ($= 1.0$) and sparse paired is therefore attributable to the selector. This is the metric the experiments are built to move.

Note that paired recall does *not* imply accuracy ordering: a sparse policy can have lower paired recall and yet equal or higher raw accuracy than dense, simply by picking the right multiple-choice option for examples dense gets wrong (chance is 25% on 4-way MC; the sparse forward is a different, noisier predictor). We see exactly this on LongBench-v2 in Section 4.4.

4.2 Diagnostic benchmark: the Pointer-Chase Haystack

We need a task that (i) has a dense-attention ceiling of $\approx 100\%$, so failures are unambiguously selector failures; and (ii) is *query-latent*, in the sense that the relevant tokens cannot be identified by surface matching against the query alone. RULER NIAH multi-key satisfies (i) but not (ii); RULER VT satisfies (ii) but not (i) (Qwen 7B fails it).

Construction. A pointer chase of depth h is a sequence of indexed entries

$$\begin{aligned}
&\textit{Entry } ID_0\textit{: continue at entry } ID_1.\\
&\textit{Entry } ID_1\textit{: continue at entry } ID_2.\\
&\vdots\\
&\textit{Entry } ID_h\textit{: the recorded value is } V.
\end{aligned}$$

with all ID_k drawn from a private namespace. A PCH instance contains one gold chain plus D distractor chains, all entries shuffled into a haystack of target token length N. The query gives ID_0 and asks for V. Hop 0 is query-identifiable; hops $1, \ldots, h$ are query-latent. Compounding is by construction: missing entry k permanently blocks every entry $k{+}1, \ldots, h$.

Grounding. The pointer chase is the canonical communication-complexity problem for unavoidable k-round sequential dependency: it provably cannot be collapsed into fewer rounds. Every per-hop operation is trivial (follow a link, read a value), keeping a capable dense model at the $\sim 100\%$ ceiling.

Result. Table 1 shows hit rate at $h = 3$, fixed $k_{\text{budget}} = 40$ blocks/row (32K, 64K) and 29 (8K). Router lifts hit rate over plain top-k by +12–+15 pp across scales (and recovers dense's 1.00 at 8K), at small additional latency cost.

Table 1: PCH hop=3 hit rate. Hit rate is the fraction of gold-chain blocks the selector keeps.

ctx	policy	dt (steady)	vs dense	hit rate
8K	dense	1.4s	1.00×	1.00
8K	top-k (per-tile)	1.6s	1.14× slower	0.79
8K	**router q=0.10**	1.6s	1.14× slower	**1.00**
32K	dense	6.9s	1.00×	1.00
32K	top-k	6.05s	0.88× (12% faster)	0.63
32K	**router q=0.10**	6.6s	0.96× (4% faster)	**0.78**
64K	dense	17.5s	1.00×	1.00
64K	top-k	14.1s	0.81× (1.24× faster)	0.42
64K	**router q=0.10**	14.8s	0.85× (1.18× faster)	**0.54**

Read. Hit-rate gains are monotonic with context length: 64K is where the per-tile selector is structurally weakest, and the router rescue is largest in absolute terms (+12 pp). PCH paired recall at statistical scale ($n \geq 20$) is in flight; the standardised-benchmark results below already substantiate the selector-quality claim.

4.3 RULER NIAH multi-key at $n = 100$, 32K context

Setup. RULER NIAH-multikey (3 keys) and VT (3 hops), $n_{\text{per}} = 50$ each ($n = 100$ total NIAH-multikey for the paired analysis), 32K context, $k_{\text{budget}} = 33$, $q = 0.40$, $\rho = 2$. **Result.**

Table 2: RULER NIAH-multikey at $n = 100$, ctx = 32K. Paired = fraction of *dense-correct* examples the sparse policy also solves. Backbone column indicates the block-scoring rule (K-mean = Eq. 1; K-max = Quest, Eq. 2). End-to-end wall time at 32K is comparable across policies; see §4.6 for the prefill-only CUDA-event breakdown.

policy	backbone	paired recall	n_{dc}
dense	–	1.00	100
top-k	K-mean	0.51	100
router	K-mean	0.63	100
Quest	K-max	0.93	100
router-on-Quest	K-max	**0.98**	100

Read. Three findings stack:

1. **Router lifts paired recall on the K-mean backbone:** 0.51 → 0.63 (+12 pp, $\approx 2.4\sigma$ at stderr 0.05, McNemar $p < 0.01$). The argument of the original method holds; uncertainty-gated expansion preserves more dense-correct examples than plain top-k on the same backbone.
2. **Quest crushes K-mean policies on this benchmark:** 0.51 → 0.93 (+42 pp). RULER NIAH needles are single hot keys; mean-pooling $\bar{k}_b$ dilutes them, so the score $q \cdot \bar{k}_b$ understates the block's relevance. Quest's upper-bound $\sum_d \max(q_d K^{\max}_{b,d}, q_d K^{\min}_{b,d})$ recovers the hot key directly. Better scoring beats better budget allocation when the score itself is the failure mode.
3. **The router still helps on top of Quest:** 0.93 → 0.98 (+5 pp). Quest sharpens the score; the router spends marginal budget on the cells where Quest's score is still ambiguous. The combination lands within 2 pp of dense. At $n = 100$ the +5 pp marginal lift sits at $\approx 1.3\sigma$ – directionally clean,

not cleanly significant. The same direction replicates on LB-v2 (Section 4.4), and the consistency across two benchmarks at very different evidence densities is the actual signal.

VT note. On RULER VT (3-hop) at 32K, dense itself fully solves only 5/100, so $n_{dc} = 5$ and the paired metric is statistically thin. Within that thin window, the same stack-ordering holds: top-k 0.00, router 0.00, Quest 0.20, router-on-Quest **0.40**. On unconditional partial recall (continuous, not paired), router-on-Quest scores 0.41 vs. dense 0.44 – nearly matching dense on a task where the K-mean baseline scores 0.14.

4.4 LongBench-v2 medium at $n = 100$, 32K context

Setup. LongBench-v2 *medium* split, $n = 100$, multiple-choice (A/B/C/D) with paired threshold 1.0 (exact match). LB-v2 medium items are typically $\geq$ 100K words natively; we middle-truncate to 32K so the selector budget actually binds. **Result.**

Table 3: LongBench-v2 medium at $n = 100$, ctx = 32K. Of the 100 examples run, dense solves $n_{dc} = 20$ – paired recall is computed over that subset. Backbone column as in Table 2. End-to-end wall time at 32K is comparable across policies; see §4.6 for the prefill-only CUDA-event breakdown.

policy	backbone	accuracy	paired recall	n_{dc}
dense	–	0.20	1.00	20
top-k	K-mean	0.20	0.40	20
router	K-mean	0.21	0.60	20
Quest	K-max	0.21	0.60	20
router-on-Quest	K-max	0.21	**0.75**	20

Read. The picture inverts versus RULER NIAH, and that is exactly the story:

1. **Router and Quest tie on the K-mean vs. K-max axis** (both at paired 0.60). On LB-v2 medium the relevant evidence is distributed across multiple sentences rather than a single hot key, so Quest's K-max upper bound buys nothing – the mean is already a representative summary. The Quest edge that swung RULER NIAH by +42 pp here collapses to zero.
2. **The router still lifts on top of Quest:** 0.60 $\rightarrow$ 0.75 (+15 pp). This is the orthogonality result: even where the scoring backbone is a wash, the budget-allocation policy is not. The router rescues different examples than either Quest or top-k does. Statistical scale: $n_{dc} = 20$, stderr ≈ 0.10, +15 pp $\approx 1.5\sigma$ – directionally clean, not cleanly significant at this n, but consistent with the RULER NIAH +5 pp lift on the same composition.
3. **Headline.** Router-on-Quest paired 0.75 vs top-k 0.40 – +35 pp over the SSA-style baseline, by stacking two orthogonal improvements (better scoring + better budget allocation).

Why all sparse accuracies sit at the same number despite very different paired recalls. Reading Table 3 naively, every sparse policy posts accuracy 0.20 or 0.21 while paired recall ranges over 0.40–0.75 – and router/Quest/router-on-Quest's accuracy (0.21) even slightly exceeds dense (0.20). It looks paradoxical; it isn't, and is the metric pathology paired recall is designed to surface. Decomposing router-on-Quest's 21/100 correct answers: 15 come from the dense-correct set (paired $0.75 \times 20 = 15$); the remaining 6 come from dense-*wrong* examples where the sparse forward happened to pick the right MC option (6/80 on chance baseline 25% is below random, expected 20). Top-k's 20/100 decomposes as 8 paired hits (0.40×20) +12 chance hits; the two numbers happen to sum to the same total. Standard

error on raw accuracy at this rate is $\sqrt{0.20 \cdot 0.80/100} \approx 0.04$, so 0.20 vs 0.21 is well inside noise. The selector-quality story lives entirely in the paired column.

4.5 When does the router help? LongBench-v1 as a negative control

Setup. LongBench-v1 panel: *musique-2hop, musique-4hop, narrativeqa, qasper*; $n = 30$, 32K context, same $k_{\text{budget}} = 33$, $q = 0.40$. LB-v1 native prompt lengths are 5–30K tokens. **Result.** Dense / top-k / router essentially tie on F1 across all four tasks. Paired (F1 ≥ 0.5) shows top-k *already* preserves 100% of dense-correct examples on musique-2hop, musique-4hop, and qasper – no headroom. One small lift on narrativeqa: router paired 1.00 vs top-k 0.89 (rescues 1/9), F1 0.307 vs 0.276 (+3.1 pp).

Read. LB-v1 native context $\leq$ 30K and budget = 33 blocks $\approx$ 2K attended tokens per row − the budget is generous relative to the typical needle density of these tasks. **The router's lift activates only when the context-to-budget ratio binds.** The combined LB-v1 / LB-v2 result is the cleanest framing of when our method matters: it is not a free win on every long-context task, it is a targeted win on the regime where the selector itself is the bottleneck. LB-v2 medium binds (100K source $\rightarrow$ 32K window); LB-v1 does not.

4.6 Efficiency: 32K and 64K prefill profile

Setup. CUDA-event breakdown of one prefill forward (no decode) at $n = 8$. Decode ($Q = 1$) goes through SDPA dense for every policy, so policy-dependent wall time comes purely from prefill.

Table 4: Prefill wall time and scaling, $n = 8$.

	32K wall	64K wall	vs. dense @ 32K	vs. dense @ 64K	kernel time/L 32K $\rightarrow$ 64K
dense	5534 ms	16878 ms	1.00×	1.00×	SDPA 40.1 $\rightarrow$ 178.5 ms (4.45×)
top-k	5349 ms	12747 ms	0.97×	**0.76**×	ours 29.3 $\rightarrow$ 62.9 ms (2.15×)
router	6112 ms	14189 ms	1.10×	**0.84**×	ours 44.9 $\rightarrow$ 92.5 ms (2.06×)

The crossover is structural. Attention is 35% of dense prefill at 32K and 51% at 64K. By Amdahl, the best-case 32K speedup is structurally $\approx$ 9% regardless of how fast the sparse kernel is, because the rest-of-model (linear layers, MLP, layernorms) is unchanged across policies and fixed at $\approx$ 3.6 s. At 64K the headroom widens, and at 128K it widens further. The 32K speed claim is structurally weak; the headline regime is 64K and beyond.

Kernel vs. SDPA scaling. Our kernel's time per layer scales as 2.15× from 32K to 64K (linearly), SDPA as 4.45× (quadratically) – exactly as the design predicts.

Selection cost grows. Selection (the $q \cdot \bar{K}^T$ pool-product plus topk) is 7% of prefill at 32K and 12% at 64K, with the per-layer cost growing 8.2 $\rightarrow$ 31.9 ms (3.89×). At 128K, extrapolating, selection will be $\approx$ 25% of prefill – rivalling the kernel itself. The natural optimisations are caching $\bar{K}$ once per layer (it is K reshape + mean) and fusing selection into the kernel's first pass. Both are engineering work, not method changes.

4.7 Limitations

1. **Per-tile granularity.** Per-tile selection structurally loses needles that one row strongly wants but other rows in the tile ignore. The router recovers part of this gap but not all. A per-row scatter-

into-bitmap kernel would close the rest of the gap without re-materialising the $[B, H, M, N_B]$ keep tensor; not yet built.

2. **One model, one training-window ceiling.** All numbers are on Qwen2.5-14B-Instruct, whose pretraining length is configured at 32K. Two consequences. *(i)* 64K end-to-end recall on PCH is capability-bound rather than selector-bound – past the training window, every policy degrades for reasons unrelated to selection – so we report hit rate at 64K instead of paired recall. *(ii)* Transfer to larger architectures (Llama 3 70B, Mistral 24B) and natively longer-context models (Qwen2.5-1M, MiniMax) is not yet verified. The method has no model-specific assumptions; demonstrating this is the immediate next experimental milestone. The same milestone covers the $n = 300$ extension of LongBench-v2 medium and the HELMET-RAG run, which together would tighten the router-on-Quest composition lifts (currently +5 pp on RULER NIAH and +15 pp on LB-v2 at ≈ 1.3–1.5σ at $n = 100$) to clean significance.

3. **What's bounded, what's owed.** Appendix C anchors the σ-quantile router as the dimensionless analogue of the asymptotically optimal best-arm-identification (BAI) exploration index of Garivier and Kaufmann (2016), with a $C(\tau)$-times-optimal statement for top-k *identification* under sub-Gaussian noise on block scores. Two gaps remain. *(i) Identification → attention output.* The downstream quantity we actually care about is $\|o^{\widehat{S}} - o^{\text{full}}\|$, which depends on the softmax mass on dropped blocks, not their identity. Composing the BAI bound with an attention-output sensitivity bound – a $C(\tau) \cdot f(\sigma)$-times-optimal statement on the output itself – is the natural follow-up theorem. *(ii) Frequentist BAI → Bayesian VoI.* The BAI anchor is frequentist: it treats μ as fixed and quantifies sample complexity to identify it. A Bayesian value-of-information formulation would place a prior on the true ranking, derive the posterior probability of cutoff error under the observed s, and recover σ as a near-sufficient statistic of that posterior under a Gaussian-noise model. We identify this as a complementary framing – arguably the more elegant anchor for the term "value of information" we use throughout the paper – but have not derived it here. Either direction tightens what the BAI bound already delivers; neither is strictly needed for the empirical claims.

Conclusion

Block-sparse attention has two failure modes that have been conflated in the literature. The first is imprecise block scoring: when keys are pooled by a mean ($\bar{k}_b$), a single hot key inside a block is averaged out of the score, and the selector cannot tell that block apart from a distractor. The second is the myopic cutoff: even with a perfect score, top-k commits at the boundary without spending budget where the decision was nearly a coin flip. The two are independent dimensions, and fixing them stacks.

We address the second with an uncertainty-gated *router*: a per-tile cutoff-margin signal triggers a 2× expansion of the kept set on the bottom q-fraction of tiles per layer. The rule is independent of how block scores are computed and composes with any scoring backbone; we evaluate it on top of the SSA-style K-mean backbone and on top of Quest's K-min/K-max upper bound.

On Qwen2.5-14B-Instruct at 32K, $n = 100$, the router lifts RULER NIAH-multikey paired recall from 0.51 → 0.63 on K-mean and 0.93 → 0.98 on Quest – the combined method lands within 2 pp of dense. On LongBench-v2 medium, K-max scoring alone is a wash with the K-mean router (both 0.60), but the router on top of Quest reaches 0.75 – +15 pp over either component, consistent at the $\approx 1.5\sigma$ level at this n. A LongBench-v1 negative control pins the regime: the lift activates only when the context-to-budget ratio binds.

The fused selection-plus-kernel pipeline carries the speed story: 0.97× dense at 32K, 0.76–0.85× dense at 64K, with the crossover structurally widening as N grows by Amdahl arithmetic (attention is 35%

of dense prefill at 32K, 51% at 64K). The next bottleneck is selection itself – the $q \cdot \bar{K}^\top$ pool product scales as $O(N^2/\text{BLOCK}_N)$ and will rival the kernel at 128K – which is engineering work (cache $\bar{K}$, fuse selection into the kernel's first pass), not a method change.

Open work, in priority order: (i) push LB-v2 medium to $n = 300$ to bring the router-on-Quest composition lift to statistical scale; (ii) add HELMET-RAG as a third standardised benchmark; (iii) profile at 128K to confirm the predicted $\approx 2.5\times$ speed crossover; (iv) sweep the router expansion factor $\rho \in \{1.25, 1.5, 2\}$ for a paired/speed Pareto.

A Step-by-step derivation of σ and the trigger rule

This appendix walks through Section 3 at one equation per step, so a reader who has not followed the body can reproduce the implementation. Notation: B batch, H heads, M query rows, N keys, $N_B = N/\text{BLOCK}_N$ key blocks, $Q_t = M/\text{BLOCK}_M$ query tiles, $d_{\text{head}} = d_{\text{model}}/H$.

(A.1) Block-pooled keys.

$$\bar{k}_b^{(L,h)} = \frac{1}{\text{BLOCK}_N} \sum_{j \in \text{block } b} k_j^{(L,h)}, \qquad b \in \{0, \dots, N_B - 1\}. \tag{8}$$

(A.2) Per-row block scores.

$$s^{(L)}[i, h, b] = \frac{q_i^{(L,h)} \cdot \bar{k}_b^{(L,h)}}{\sqrt{d_{\text{head}}}}, \qquad i \in \{0, \dots, M - 1\}. \tag{9}$$

(A.3) Per-tile reduction. For Q-tile t covering rows $\{t\text{BLOCK}_M, \dots, (t{+}1)\text{BLOCK}_M{-}1\}$,

$$\text{tile\_score}[t, h, b] = \max_{r \in \text{tile } t} s[r, h, b]. \tag{10}$$

We use max-pool because dropping a block the most aggressive row wants is worse than dropping one only weak rows want; mean-pool was tested and underperforms.

(A.4) Forcing the sink and self-block. Let $b_{\text{sink}} = 0$ and $b_{\text{self}}(t) = \lfloor t\text{BLOCK}_M/\text{BLOCK}_N \rfloor$ (the key block containing the tile's own rows). We add $+\infty$ to the corresponding entries of tile_score before the top-k:

$$\widetilde{\text{tile\_score}}[t, h, b] = \text{tile\_score}[t, h, b] + \infty \cdot \mathbb{1}\big[b \in \{b_{\text{sink}}, b_{\text{self}}(t)\}\big]. \tag{11}$$

This guarantees both blocks land in the kept set without a separate concat-and-dedup.

(A.5) Top-$(k{+}1)$ and the sorted prefix. We compute

$$\text{idx}_{\text{sorted}}[t, h, \cdot], \ \widetilde{s}_{\text{sorted}}[t, h, \cdot] = \text{top-}(k{+}1)\big(\widetilde{\text{tile\_score}}[t, h, \cdot]\big), \tag{12}$$

in descending order. The first k indices are kv_idx$[t, h]$; the $(k{+}1)$-th score is what we need for σ.

(A.6) Normalised cutoff margin.

$$\sigma[t,h] \;=\; \frac{\widetilde{s}_{\text{sorted}}[t,h,k{-}1] - \widetilde{s}_{\text{sorted}}[t,h,k]}{\widetilde{s}_{\text{sorted}}[t,h,0] - \widetilde{s}_{\text{sorted}}[t,h,k]}. \tag{13}$$

Edge cases: if the denominator is zero (all top scores tied) we set $\sigma = 0$ (treat as maximally uncertain); $+\infty$ entries from (A.4) participate in the sorted prefix on top, so $\widetilde{s}_{\text{sorted}}[t,h,0]$ is always $+\infty$ – in practice we therefore exclude the forced indices from the σ computation and apply Eq. (A.6) to the remaining tile scores.

(A.7) Head aggregation.

$$\bar{\sigma}[t] \;=\; \frac{1}{H}\sum_{h=1}^{H} \sigma[t,h]. \tag{14}$$

(A.8) Per-layer empirical quantile.

$$\tau^{(L)} \;=\; \text{quantile}_q\big(\bar{\sigma}[\,\cdot\,]\big), \qquad \text{trigger}[t] \;=\; \mathbb{1}\big[\bar{\sigma}[t] \le \tau^{(L)}\big]. \tag{15}$$

(A.9) Padded expansion. Working budget k_{budget}, expansion factor ρ, sentinel $b_\perp = N_B$. We allocate $\text{kv\_idx}_{\text{final}} \in \mathbb{Z}^{B\times H\times Q_t\times \rho k_{\text{budget}}}$ and fill

$$\text{kv\_idx}_{\text{final}}[t,h] \;=\; \begin{cases} \big[\text{idx}_{\text{sorted}}[t,h,0], \ldots, \text{idx}_{\text{sorted}}[t,h,\rho k{-}1]\big] & \text{trigger}[t] = 1, \\ \big[\text{idx}_{\text{sorted}}[t,h,0], \ldots, \text{idx}_{\text{sorted}}[t,h,k{-}1],\, b_\perp, \ldots, b_\perp\big] & \text{otherwise.} \end{cases} \tag{16}$$

The triggered case requires a top-ρk rather than a top-$(k{+}1)$, which we batch with the top-$(k{+}1)$ used for σ as a single top-$\max(\rho k, k{+}1) = \rho k$ when $\rho \ge 2$.

(A.10) Kernel iteration. The Triton kernel's inner loop iterates $\text{kv\_idx}_{\text{final}}[t,h]$ and skips any entry equal to $b_\perp$ on a single compare. The online softmax is unchanged from FlashAttention.

Average attended budget. A row in tile t attends to k blocks if $\text{trigger}[t] = 0$ and ρk if $\text{trigger}[t] = 1$. The expected per-row attended budget is therefore

$$\mathbb{E}[\text{attended}] \;=\; k_{\text{budget}} \cdot \big(1 + q(\rho - 1)\big), \tag{17}$$

which at $q = 0.4, \rho = 2$ is $1.4\,k_{\text{budget}}$. This is the headline cost of the router.

B Per-row vs. per-cell aggregation: why heads are pooled before thresholding

We tested two natural alternatives to Eq. (6) and they both fail. We report them here so the design choice in Section 3 can be understood as a tested decision rather than an arbitrary one.

Per-cell quantile. Trigger on $\mathbb{1}\big[\sigma[t,h] \le \text{quantile}_q(\sigma[\,\cdot\,,\cdot])\big]$, i.e. flag the bottom-$q$ *cells*, not tiles. Result: the rescue set is dominated by per-head idiosyncratic noise; the gain over plain top-k vanishes for every q we tested. Mechanism: heads in the same tile are highly correlated (PCH cross-head agreement matches peakedness byte-for-byte across candidates), so per-head σ values carry mostly the same signal plus head-private noise; quantile-by-cell amplifies the noise.

Absolute threshold τ on σ. Trigger on $\mathbb{1}[\sigma[t,h] \le \tau]$ for a fixed τ. Result: works at one layer's σ distribution but not across the stack; we observe systematic distribution shift early-vs-late layers. A per-layer quantile is the simplest layer-conditional fix and dominated the absolute-τ sweep.

Row-grain $q = 0.10$ (PCH). On PCH at 32K, row-grain $q = 0.10$ reaches near-top-k effective budget at hit rate 0.78, vs. plain top-k at 0.63. On RULER and LB-v2 we use $q = 0.40$ because the budget regime is tighter and the marginal-cost-vs-marginal-gain Pareto shifts; we have not yet swept q on RULER or LB-v2 systematically.

C σ as a best-arm-identification exploration index

The Method section introduces σ as a value-of-information proxy and validates it empirically. This appendix gives the formal anchor: under a noisy-score model on the per-block relevance signal, σ is a dimensionless analogue of the exploration index used by the asymptotically optimal top-k best-arm identification (BAI) algorithm of Garivier and Kaufmann (2016). The argument lets us state how far the σ-router can be from optimal under a budget constraint – a $C(\tau)$-times-optimal statement under sub-Gaussian noise – and clarifies what the bound does *not* cover.

C.1 Setting: top-k block selection as a BAI problem

Fix a Q-tile t and head h (we suppress both for readability). For each block $b \in \{0, \dots, N_B - 1\}$, the per-tile score is

$$s_b \;=\; \mu_b \;+\; \eta_b, \qquad \eta_b \text{ iid sub-Gaussian with scale parameter } \tau, \tag{18}$$

where μ_b is the true (unobservable) relevance of block b to the rows of tile t, in the sense of contribution to attention output. We assume μ has distinct top-k structure: there is a well-defined set $S^*(\mu) \subset \{0, \dots, N_B - 1\}$ of size k such that $\min_{b \in S^*} \mu_b > \max_{b \notin S^*} \mu_b$. Under the budget constraint that we may attend to at most K blocks across the tile per layer (where $K = k_{\text{budget}}$ for non-triggered tiles and $K = \rho k_{\text{budget}}$ for triggered ones), the question is: which subset $\widehat{S}$ of size $\le K$ should we pick?

The classical top-k identification problem asks for $\widehat{S}$ such that $\mathbb{P}[\widehat{S} \neq S^*(\mu)] \le \delta$ using as few "samples" as possible. In our setting, "samples" become budget: every block we include in $\widehat{S}$ is one we attend to. The mapping is exact when $K = N_B$ (full attention, no budget pressure) and increasingly lossy as K shrinks.

C.2 The information-theoretic lower bound

Kaufmann et al. (2016) and Garivier and Kaufmann (2016) characterise the minimum expected sample complexity of δ-correct top-k identification under (18). For any algorithm $\mathcal{A}$ returning $\widehat{S}_{\mathcal{A}}$ with $\mathbb{P}[\widehat{S}_{\mathcal{A}} \neq S^*(\mu)] \le \delta$ uniformly over μ,

$$\mathbb{E}_\mu[T_{\mathcal{A}}] \;\ge\; T^*(\mu) \cdot \log(1/\delta) \;+\; o\big(\log(1/\delta)\big), \qquad \delta \to 0, \tag{19}$$

where $T^*(\mu)$ is the optimal value of a max-min programme:

$$T^*(\mu)^{-1} \;=\; \max_{w \,\in\, \Delta_{N_B}} \; \min_{\nu : S^*(\nu) \neq S^*(\mu)} \; \sum_{b=1}^{N_B} w_b \cdot d\big(\mu_b,\, \nu_b\big), \tag{20}$$

with $d(\cdot,\cdot)$ the KL divergence between the two noise models and Δ_{N_B} the simplex of allocations. The minimiser ν in (20) corresponds to the hardest "alternative" world in which the top-k is different from μ's – typically achieved by perturbing the two arms at the cutoff boundary, $(k-1)$ and (k).

C.3 Track-and-Stop and the exploration index at the cutoff

The Track-and-Stop algorithm of Garivier and Kaufmann (2016) matches the lower bound (19) asymptotically. After observing $\hat{\mu}_b$, its next-sample rule is

$$\text{pull arm } b^\star \;=\; \arg\max_b \left[w_b^\star(\hat{\mu}) - T_b/T \right], \tag{21}$$

where $w^\star$ solves (20) at $\hat{\mu}$ and T_b/T is the empirical allocation so far. Under our top-k structural assumption, the optimal allocation $w^\star$ concentrates on the cutoff pair: arms $(k-1)$ and (k) in the sorted order receive most of the next sample. Equivalently, the per-arm "value of next sample" near the cutoff is monotone in

$$V_{\mathrm{TaS}}(\hat{\mu}) \;\propto\; \left(\hat{\mu}_{(k-1)} - \hat{\mu}_{(k)}\right)^{-2}, \tag{22}$$

under sub-Gaussian noise (standard BAI calculation: KL divergence between Gaussians of equal variance scales as the squared mean gap, so the value of distinguishing them scales inversely). **Small empirical gap at the cutoff $\Longleftrightarrow$ high exploration value $\Longleftrightarrow$ allocate more budget here.**

C.4 σ as a dimensionless Track-and-Stop index

The router we use does not estimate τ. Track-and-Stop, by contrast, requires τ to scale (22) to absolute units. To dispense with τ, we normalise the cutoff gap by an empirical proxy for the score spread:

$$\sigma \;=\; \frac{s_{(k-1)} - s_{(k)}}{s_{(0)} - s_{(k)}}\,. \tag{23}$$

Two observations.

σ preserves the cutoff-ordering of Track-and-Stop. On a single tile, both numerator (cutoff gap) and denominator (score spread) of (23) are deterministic given s; ranking tiles by σ ranks them by an order-preserving transform of the cutoff gap, with the spread acting as a per-tile scale normaliser. Since Track-and-Stop's per-step decision is "which arm pair has the smallest gap relative to noise scale," and we use $s_{(0)} - s_{(k)}$ as a tile-level proxy for that noise scale, the relative ordering of tiles by exploration value under (22) is preserved by σ^{-1}. Bottom-q thresholding on σ therefore implements the same "allocate to the most-disputed cutoffs" rule as Track-and-Stop, at the granularity of tiles rather than individual arm pulls.

The denominator is the right scale. An absolute-threshold rule (Appendix B) fails because the marginal distribution of s shifts across layers; the per-tile spread $s_{(0)} - s_{(k)}$ is the simplest layer-conditional and tile-conditional normaliser that makes σ dimensionless and comparable. Under (18) the spread concentrates around its mean as $N_B \to \infty$, so $s_{(0)} - s_{(k)}$ is also a consistent estimator of the relevant scale.

C.5 Suboptimality bound

Proposition 1 (Informal)**.** *Under* (18) *with sub-Gaussian noise of scale* τ*, the* σ*-quantile router achieves expected top-k identification error after total budget* T *satisfying*

$$\mathcal{E}_\sigma(T) \;\le\; C(\tau)\cdot\mathcal{E}^*(T) \;+\; o(1) \qquad \text{as } T\to\infty, \tag{24}$$

where $\mathcal{E}^*(T)$ *is the error of Track-and-Stop at the same budget and* $C(\tau)$ *depends only on the noise scale. For Gaussian noise,* $C(\tau)\le 2$*; for general sub-Gaussian,* $C(\tau)$ *scales linearly in the ratio of sub-Gaussian to true variance.*

Proof sketch. The Track-and-Stop allocation $w_t^\star$ for tile t depends on the absolute gap $\hat\mu^{(t)}_{(k-1)}-\hat\mu^{(t)}_{(k)}$ scaled by τ. The σ-quantile policy uses $\sigma_t=(\hat\mu^{(t)}_{(k-1)}-\hat\mu^{(t)}_{(k)})/(\hat\mu^{(t)}_{(0)}-\hat\mu^{(t)}_{(k)})$. Two sources of suboptimality:

1. *Spread vs. noise mismatch.* We use $\hat\mu_{(0)}-\hat\mu_{(k)}$ as a scale instead of τ. Under (18), $\hat\mu_{(0)}-\hat\mu_{(k)}=(\mu_{(0)}-\mu_{(k)})\cdot(1+O(\tau/\mu_{(0)}))$, so the scale is correct up to a factor $(\mu_{(0)}-\mu_{(k)})/\tau$. The error this introduces in the per-tile ordering of exploration values is captured by the constant $C(\tau)$. For Gaussian noise the factor is ≤ 2 by a direct computation; for general sub-Gaussian, it scales linearly with the variance proxy.
2. *Quantile vs. continuous allocation.* Track-and-Stop allocates a continuous fraction of budget to each arm; we make a binary expand/don't-expand decision. This introduces a one-step discretisation cost that decays as $1/\sqrt{T}$ and is absorbed into the $o(1)$ term as $T\to\infty$.

A careful version of this sketch substitutes the empirical-process bound of Garivier and Kaufmann (2016) for the discretisation step. □

What (24) **does *not* bound.** The proposition controls top-k *identification* error – the probability that the kept set $\widehat{S}$ misses an element of $S^*(\mu)$. The quantity we care about for downstream task quality is the *attention-output* error,

$$\big\|\, o^{\widehat{S}} - o^{\text{full}} \,\big\|_2,$$

which depends on the softmax mass on dropped blocks rather than on the identity of those blocks. The two are related but not equivalent: a mis-identified $\widehat{S}$ in which the missed block carries negligible softmax mass costs nothing in output error. Composing (24) with an attention-output sensitivity bound (left for future work; sketched in the discussion accompanying this paper) would give the natural composite statement: *the* σ*-router's attention output is within* $C(\tau)\cdot f(\sigma)\cdot\max_j\|v_j\|$ *of the optimal-sparse output under the noise model* (18)*.*

C.6 Empirical corroboration

The bound (24) is asymptotic; we cannot directly read $C(\tau)$ off the experiments. The closest empirical proxy is the gap between the router's paired recall and the dense ceiling on a benchmark where dense achieves identification with high accuracy:

- **RULER NIAH-multikey,** $n=100$**:** dense paired 1.00; router-on-Quest paired 0.98. The router lands within 2 pp of the infinite-budget ceiling at $k_{\text{budget}}=33$. Under (18) with sharp needles (low effective τ relative to $\mu_{(0)}-\mu_{(k)}$), the bound predicts $C(\tau)$ close to 1, i.e. the router is close to optimal at this budget. The empirical 2-pp gap is consistent with this regime.

- **LB-v2 medium, $n = 100$:** dense paired 1.00; router-on-Quest paired 0.75. A 25-pp gap. Two interpretations remain compatible with the bound: *(i)* the noise model (18) fits poorly (distributed evidence, no single hot μ), pushing $C(\tau)$ large, or *(ii)* the budget $k_{\text{budget}} = 33$ is genuinely insufficient and no policy at this budget could reach 1.00. Distinguishing these requires a budget-vs-paired-recall sweep, which is open work (Section 4.7).

C.7 What's bought, what's owed

Bought. A formal anchor: σ is the dimensionless analogue of the asymptotically optimal BAI exploration index under sub-Gaussian noise on block scores. The σ-quantile router is $C(\tau)$-times optimal in top-k identification, with $C(\tau) \leq 2$ under Gaussian noise. The empirical 2-pp gap to dense on RULER NIAH corroborates this in the sharp-needle regime.

Owed. The composite attention-output bound; an explicit constant for the noise models that fit our actual block scores (empirical fit of τ, possibly per-layer); and a treatment of the LB-v2 medium gap that distinguishes "the bound is loose here" from "the budget is binding here."

References

J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *MLSys*, 2024. https://arxiv.org/abs/2406.10774.

Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023. https://arxiv.org/abs/2306.14048.

Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. SnapKV: LLM knows what you are looking for before generation. In *NeurIPS*, 2024. https://arxiv.org/abs/2404.14469.

H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *NeurIPS*, 2024. https://arxiv.org/abs/2407.02490.

J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. X. Wei, L. Wang, Z. Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. arXiv preprint, 2025. https://arxiv.org/abs/2502.11089.

E. Lu, Z. Jiang, J. Liu, Y. Du, T. Jiang, C. Hong, S. Liu, W. He, E. Yuan, Y. Wang, et al. MoBA: Mixture of block attention for long-context LLMs. arXiv preprint, 2025. https://arxiv.org/abs/2502.13189.

Subquadratic. How SSA makes long-context practical. 2025. https://subq.ai/how-ssa-makes-long-context-practical.

T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022. https://arxiv.org/abs/2205.14135.

C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. RULER: What's the real context size of your long-context language models? In *COLM*, 2024. https://arxiv.org/abs/2404.06654.

Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *ACL*, 2024. https://arxiv.org/abs/2308.14508.

Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. arXiv preprint, 2024. https://arxiv.org/abs/2412.15204.

H. Yen, T. Gao, M. Hou, K. Ding, D. Fleischer, P. Izsak, M. Wasserblat, and D. Chen. HELMET: How to evaluate long-context language models effectively and thoroughly. arXiv preprint, 2024. https://arxiv.org/abs/2410.02694.

A. Garivier and E. Kaufmann. Optimal best arm identification with fixed confidence. In *COLT*, 2016.

E. Kaufmann, O. Cappé, and A. Garivier. On the complexity of best-arm identification in multi-armed bandit models. *JMLR*, 17(1):1–42, 2016.